

Lecture 7 - Markov Jump Processes and their Simulation

1 - Markov Jump Process

So far, we have been learning about Markov processes that are continuous in both their state and in time. **Markov Jump Processes** are continuous-time processes with a discrete state space $\mathcal{S} = \{\phi_1, \phi_2, \dots, \phi_n\}$. The number of states may be finite or infinite. At any point in time, the system occupies some state $x(t) \in \mathcal{S}$.

Here are some examples:

1) A chemical reaction $A \rightleftharpoons B$.

State space $\mathcal{S} = \{A, B\}$, $n = 2$.

2) A random walk on a 1D-lattice

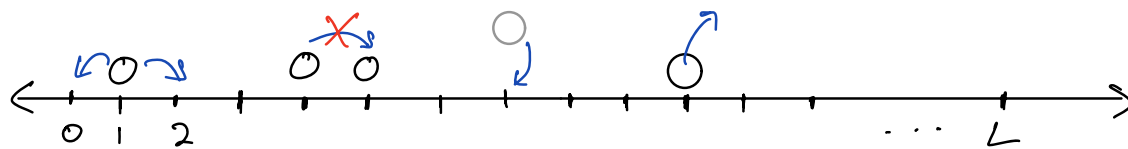
State at any time is the particle position.

Thus, $\mathcal{S} = \mathbb{Z}$, $n = \infty$



3) A collection of particles hopping on a finite 1D lattice of size L , with birth, death, and hard-core exclusion.

You can think of this as a microtubule in a solution of microtubule-binding proteins



Each state is represented by a vector of occupancy numbers

$$x(t) = (n_1(t), n_2(t), \dots, n_L(t)), \text{ where } n_i = 0 \text{ or } 1.$$

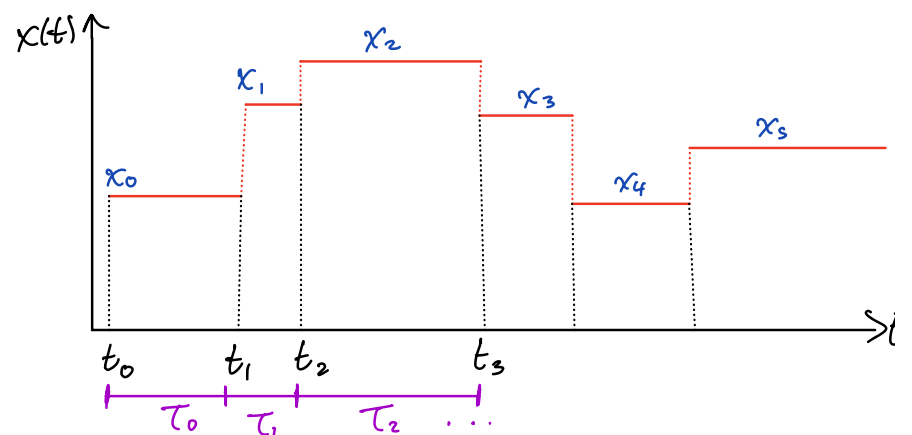
Thus $\mathcal{S} = \{0, 1\}^L$.

A trajectory of a Markov process is specified by a **sequence of states** $\{x_i\}$ and a corresponding **sequence of dwell times**:

trajectory: $((x_0, \tau_0), (x_1, \tau_1), \dots, (x_F, \tau_F), t_F)$, where $x_i \in \mathcal{S}$

The "jumps" occur at times

$$t_i \equiv t_{i-1} + \tau_{i-1}, \text{ for } i > 0.$$



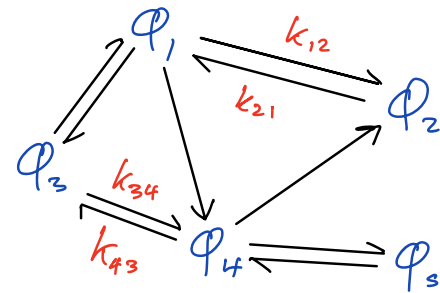
As discussed in lecture, the state occupation probabilities follow a master equation

$$\partial_t P(\varphi_i, t) = \sum_{j \neq i} W(\varphi_j \rightarrow \varphi_i) P(\varphi_j, t) - W(\varphi_i \rightarrow \varphi_j) P(\varphi_i, t)$$

where $W(\varphi_j \rightarrow \varphi_i)$ is the transition rate from state φ_j to state φ_i , defined in terms of the conditional probability

$$\lim_{\Delta t \rightarrow 0} \frac{P(\varphi_i, t + \Delta t | \varphi_j, t)}{\Delta t} = W(\varphi_j \rightarrow \varphi_i) \quad \text{for } i \neq j.$$

In the context of chemical reactions, the state space is often represented as a graph, with arrows denoting allowed transitions, and the corresponding rates indicated on the arrows ($k_{ij} \equiv W(\varphi_i \rightarrow \varphi_j)$).



2- Simulating a Markov Jump Process

2.1 Dwell times and transition probabilities

To simulate a Markov jump process requires a rule for choosing the dwell time and the next state transition given the current state.

Dwell times: We showed in lecture that the dwell times are exponentially distributed according to the escape rate

$$r(\varphi_i) = \sum_{j \neq i} W(\varphi_i \rightarrow \varphi_j),$$

so that

$$\tau_i \sim \text{Exp}(r(\varphi_i)), \quad P_{\tau_i}(t) \propto r_i e^{-r_i t}$$

Next state: We showed in lecture that, upon escaping state i , the probability that the system lands in state j is:

$$P_{i \rightarrow j} = \frac{W(\varphi_i \rightarrow \varphi_j)}{r(\varphi_i)}$$

Together, these define an algorithm to exactly sample the trajectories of a jump process, often called the **Gillespie algorithm** (though it is much older than Gillespie).

2.3 The Gillespie algorithm (Direct method)

The Gillespie algorithm, developed by Doob and others in ≈ 1945 and popularized by Gillespie in 1976-77, generates the next state and jump time of a Markov jump process as follows:

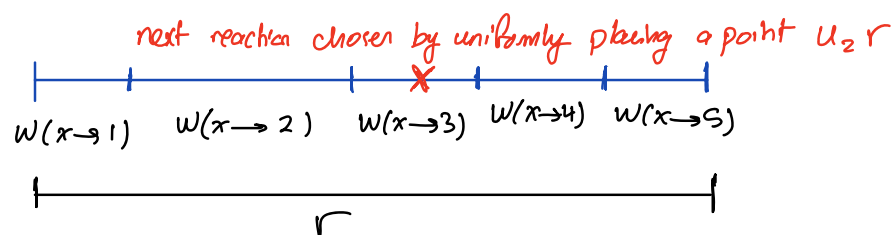
- 1) Initialize the time and state variables $t = t_0$ and $x = x_0$.
- 2) Compute the escape rate $r = \sum_{\phi \neq x} w(x \rightarrow \phi)$
- 3) Draw two independent R.V.s uniformly on $[0, 1]$, called u_1 and u_2 .
- 4) Time update: set $t = t + \tau$, where $\tau \equiv \frac{1}{r} \log(1/u_1)$
- 5) State update: set $x = \phi_j$, where

$$j = \min_l \left[\sum_{n=1}^l w(x \rightarrow \phi_n) \geq u_2 r \right]$$
- 6) Return to step 2.

Why does this work?

$\Rightarrow$ For step (4), check that $\frac{1}{r} \log(1/u_1) \sim \text{Exp}(r)$.

$\Rightarrow$ For step (5), we wish to pick the next state with a probability proportional to the rate of jumping from x to that state. See the below picture:



$\Rightarrow$ Why are the dwell time and next reaction independent? Go back to today's lecture notes

and check that the joint density $P(\tau, j)$ is $P(\tau, j) = w(x \rightarrow j) \exp\left[-\sum_l w(x \rightarrow l) \tau\right]$

This algorithm is exact and is sometimes called the "direct" method. It is "rejection free", in the sense that every random number generation is associated with a state transition. It is an example of a "kinetic Monte Carlo" algorithm.

In many situations (e.g. large reaction networks, many particles on a lattice), the number of possible state transitions is very large. This means the escape rates are very large, and the dwell times are very short, so very many iterations of the algorithm are needed to elapse a reasonably long time interval. It also means that computation of the escape rates is

itself costly as it takes a time which scales linearly with the number of possible transitions. There are some optimizations that can be done when many of the reactions are independent (e.g. far-away particles on a lattice). These rely on an alternative description of a Markov jump process.

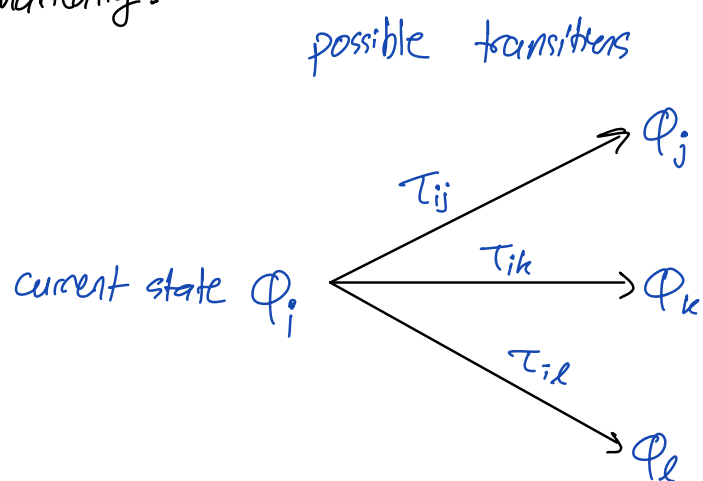
2.3 "First-reach" and "Next-reach" methods

In the above, we characterized a Markov jump process in terms of the distributions governing the dwell time and the next state transition choice. An equivalent description is the following: suppose that the system is in state ϕ_i at some time. To each possible escape reaction $\phi_i \rightarrow \phi_j$ is associated a stopwatch, and a waiting time

$$\tau_{ij} \sim \text{Exp}(W(\phi_i \rightarrow \phi_j))$$

is drawn. The stop watches are started. The first time a stopwatch hits τ_{ij} for some j , reaction $\phi_i \rightarrow \phi_j$ is executed. The waiting times are then drawn anew, and the stopwatches restarted.

Schematically:



Dwell time: $\min_j (\tau_{ij})$

Next state: ϕ_p , where $p = \arg\min_j (\tau_{ij})$

To see that this is equivalent to the above, the following must be shown:

Consider a collection of n independent exponentially-distributed random variables with rates $k_1, k_2, \dots, k_n$; that is, $\{\tau_1, \dots, \tau_n\}$ with $\tau_i \sim \text{Exp}(k_i)$

(1) The minimum $\tau \equiv \min \{\tau_i\}$ is $\text{Exp}(r)$ distributed with $r = \sum_i k_i$.

(2) The probability that τ_i is the smallest among the $\{\tau_j\}$ is:

$$\mathbb{P}[\arg\min_j \{\tau_j\} = i] = k_i / r$$

For (1), it suffices to prove the statement for $n=2$, because it may then be applied

$$\text{pairwise } \min(\tau_1, \tau_2, \dots, \tau_n) = \min(\min(\tau_1, \tau_2), \tau_3, \dots, \tau_n) = \min(\min(\min(\tau_1, \tau_2), \tau_3), \tau_4, \dots, \tau_n)$$

and so on. For $n=2$, we have

$$\begin{aligned} \mathbb{P}(\min(\tau_1, \tau_2) > t) &= \mathbb{P}(\tau_1 > t) \mathbb{P}(\tau_2 > t) = \int_t^\infty ds k_1 e^{-k_1 s} \int_t^\infty ds' k_2 e^{-k_2 s'} \\ &= e^{-k_1 t - k_2 t} \\ &= 1 - \mathbb{P}(\min(\tau_1, \tau_2) < t) \end{aligned}$$

we thus identify the CDF as that of an $\text{Exp}(k_1 + k_2)$ R.V. $\square$

For (2), we use indicator variables

$$\mathbb{P}[\arg\min_j \{\tau_j\} = i] = \left\langle \prod_{p \neq i} \mathbb{1}(\tau_p > \tau_i) \right\rangle$$

where $\mathbb{1}(A) \equiv 1$ if event A occurs and zero otherwise.

The terms within the product are not independent: if I tell you $\tau_k > \tau_i$ for some k , this tells you something about the smallness of τ_i , so that $(\tau_m > \tau_i)$ is now more likely.

To resolve this, we condition on τ_i and use the tower rule $\mathbb{E}(X) = \mathbb{E}_Y[\mathbb{E}_X[X|Y]]$:

$$\begin{aligned} \mathbb{P}[\arg\min_j \{\tau_j\} = i] &= \left\langle \left\langle \prod_{p \neq i} \mathbb{1}(\tau_p > \tau_i) \mid \tau_i \right\rangle \right\rangle_{\tau_i} \\ &= \left\langle \prod_{p \neq i} \mathbb{P}(\tau_p > \tau_i) \right\rangle_{\tau_i} \\ &= \left\langle \prod_{p \neq i} e^{-k_p \tau_i} \right\rangle_{\tau_i} \\ &= k_i \int_0^\infty dt \exp\left(-\sum_{j=1}^n k_j t\right) \\ &= \frac{k_i}{\sum_{j=1}^n k_j} \end{aligned} \quad \square$$

2.3.1: "First reaction" method

Based on the above Gillespie proposed the following algorithm: at each step

draw an $\text{Exp}(w(x \rightarrow \varphi))$ variable τ_φ for every $\varphi \in \mathcal{S}$. Set $t \rightarrow t + \min \{\tau_\varphi\}$

and $x \rightarrow \arg\min_\varphi \{\tau_\varphi\}$.

This algorithm is obviously less efficient because it draws up to $|\mathcal{S}|$ random variables at each step and identifies their minimum. Its usefulness is thus mostly pedagogical.

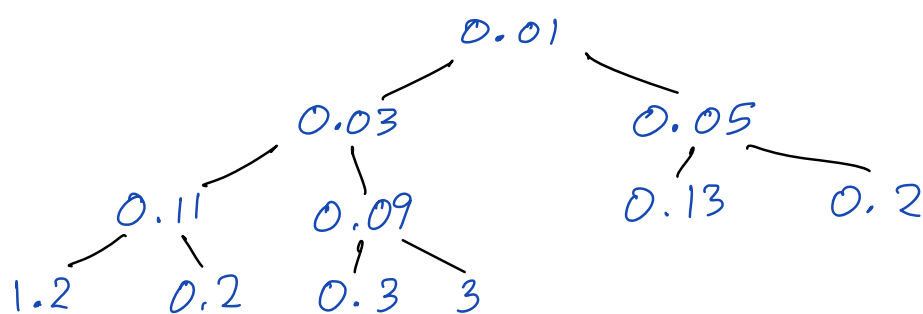
2.3.2: "Next reaction" method (Gibson and Bruck, 2000).

The idea here is to draw all the possible transition times $\{\tau_i\}$ and to store them in an efficient data structure that maintains their order.

At each step, the smallest τ_i transition is executed, and only those reactions which are affected are redrawn.

The original implementation is specialized to chemical reactions, so we list here only the main idea:

⇒ The transition times are stored in a **min heap** data structure (usually indexed)



⇒ The next reaction is always the root and can be found in $\mathcal{O}(1)$ time.

⇒ An additional data structure keeps track of the dependency structure between the elements (e.g. nearest neighbor sites on a lattice).

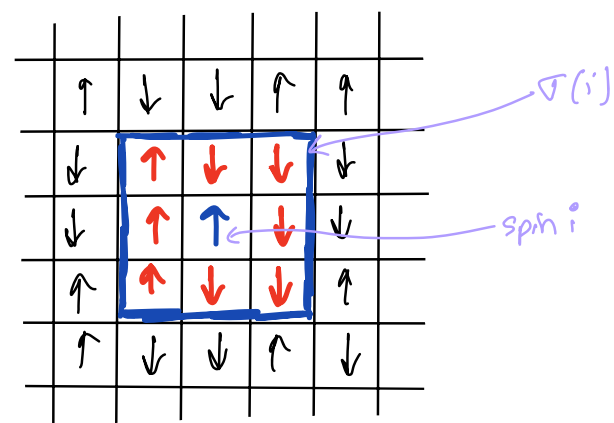
Example: spins on a lattice (e.g. a kinetic Ising model):

Consider a model with spins $s = \pm 1$ on a lattice $\mathbb{Z}^d$.

Spin i flips at a rate which is a function of s_i

and the nearest neighbors of i , denoted $\sigma(i)$:

$$\text{Flip rate} = w(\{s_j : j \in \sigma(i)\}) = w_i(\vec{s})$$



This can be simulated as follows:

1) Initialize the spin lattice. For each $i \in L^d$, draw the flip

time of spin i according to $\tau_i \sim \text{Exp}(w_i(\vec{s}))$

2) Organize the L^d pairs (i, τ_i) into a min heap sorted according

to τ_i . Create a data structure of pointers to the tree for each i .

3) Set the simulation time to $t = \tau_j$, where $j = \text{argmin}_i \{\tau_i\}$.

4) Flip spin j .

5) For each $i \in \sigma(j)$ and for j itself, redraw τ_i as $\tau_i \rightarrow t + \Delta\tau_i$, where $\Delta\tau_i \sim \text{Exp}(w_i(\vec{s}))$. Re-organize the heap. Return to step 3.

The slowest step is the time to reorganize the heap, which is $\mathcal{O}(\log N)$ instead of $\mathcal{O}(N)$.

This is a big improvement over the direct method

2.4 Discrete-time (rejection) algorithms

In cases where the number of possible reactions is very large $n \gg 1$ and all the reactions have comparable rates, it is often most efficient to use a discrete-time algorithm which converges to the jump process as $\Delta t \rightarrow 0$.

We will focus on many-body lattice models like the kinetic Ising model. For such models, a useful discrete-time algorithm is **random sequential update**, where particles/spins are randomly chosen and updated according to some probabilistic rule.

For a kinetic Ising model with flip rates $w_i(\vec{s})$ at site i , the algorithm is as follows: Pick a time step Δt . Then

- 1) Advance time $t \rightarrow t + \Delta t/N$, where $N \equiv L^d$
 - 2) Randomly select a lattice site i (**proposal**)
 - 3) With probability $w_i(\vec{s})\Delta t$, flip spin i . (**acceptance**)
 [with probability $1 - w_i(\vec{s})\Delta t$, do nothing. (**rejection**)
- Return to (1).

How do we choose Δt ? We would like for it to be as large as possible while keeping the acceptance/rejection probabilities $\in [0, 1]$. Thus $\Delta t = 1/w_{\max}$, where $w_{\max}$ is the maximum possible value of w_i .

The above describes a discrete-time **Markov Chain** that approximates the jump process as $N \rightarrow \infty$ (or as $\Delta t \rightarrow 0$). To see this, let C denote a configuration of spins, and C_i that same configuration with spin i flipped. Then the probability of a configuration obeys the following recursion relation in discrete time:

$$P(C, t + \frac{\Delta t}{N}) = \sum_{i=1}^N P(C_i, t) \frac{w_i(C_i) \Delta t}{N} + P(C, t) \left(1 - \sum_{i=1}^N \frac{w_i(C) \Delta t}{N}\right)$$

The continuous time limit follows by expanding $P(C, t + \frac{\Delta t}{N}) = P(C, t) + \partial_t P(C, t) \frac{\Delta t}{N}$, valid when $\Delta t/N$ is $\ll$ the timescale of variations in P (e.g. for $\frac{\Delta t}{N} \ll 1/w_{\max}$).

We obtain:

$$\begin{aligned} \partial_t P(C, t) &= \sum_{i=1}^N P(C_i, t) w_i(C_i) - P(C, t) w_i(C) \\ &= \sum_{C' \neq C} P(C', t) W(C' \rightarrow C) - P(C, t) W(C \rightarrow C'), \end{aligned}$$

where we have used that the only nonzero $W(C \rightarrow C')$ are those for which $C' = C_i$ for some i (all transitions involve single flips of some spin). We have thus recovered the master equation.